



Spring Boot

Complete Reference Guide

Java · Spring Framework · Spring Boot · REST API · Spring Security · JPA · JWT

1. Spring Framework Overview

What is Spring?

Spring is an open-source, lightweight Java framework for building enterprise applications. At its core, Spring provides an **IoC (Inversion of Control) Container** — a runtime that manages the lifecycle of Java objects (**beans**). Instead of your code creating dependencies manually, Spring creates and wires them for you.

Benefit	Description
Loose Coupling	Objects don't create their own dependencies — they are injected
Testability	Dependencies can be mocked or replaced easily in unit tests
Modularity	Pull in only what you need: Spring MVC, Data, Security, etc.
Boilerplate Reduction	Spring handles transactions, DB connections, and repetitive code
Convention over Config	Spring Boot makes setup near-zero with sensible defaults

Spring vs Spring Boot

Feature	Spring (Plain)	Spring Boot
Configuration	XML or Java-based, manual	Auto-configured
Server	External Tomcat setup needed	Embedded Tomcat/Jetty built in
Startup	ClassPathXmlApplicationContext	SpringApplication.run(...)
Dependencies	Manually add each one	Starter POMs handle it
Boilerplate	More setup required	Much less — run immediately

□ Use Spring Boot for modern REST APIs and microservices. Learn plain Spring to understand how it works under the hood.

2. IoC & Dependency Injection

Inversion of Control (IoC)

IoC is the principle where the **framework** controls object creation instead of your code. Without IoC you write `new Dev()`; with IoC, Spring scans, creates, and manages the object for you.

The IoC Container

When `SpringApplication.run(...)` is called, Spring:

- Scans all classes annotated with `@Component`, `@Service`, `@Repository`, `@Controller`
- Creates bean instances and stores them in the `ApplicationContext`
- Injects dependencies between beans automatically

Types of Dependency Injection

A) Constructor Injection (Recommended)

```
@Component
public class Controller {
    private final Service service;

    public Controller(Service service) { // Spring injects via constructor
        this.service = service;
    }
}
```

B) Setter Injection

```
@Autowired
public void setService(Service service) { this.service = service; }
```

C) Field Injection

```
@Autowired
private Service service; // Spring injects directly – most concise
```

@Qualifier and @Primary

When two beans implement the same interface, Spring doesn't know which to inject. Resolve with:

```
@Autowired
@Qualifier("laptop") // tells Spring exactly which bean to use
private Computer comp;
```

Or annotate one implementation with `@Primary` to make it the default choice.

3. Bean Annotations & Stereotype

All of these are specializations of `@Component` — Spring creates a bean for each.

Annotation	Layer	Purpose
<code>@Component</code>	Generic	Generic Spring-managed bean
<code>@Service</code>	Business Logic	Marks a service class; no extra behaviour over <code>@Component</code>
<code>@Repository</code>	Data Access	Marks a DAO; adds exception translation
<code>@Controller</code>	Web MVC	Handles HTTP requests, returns view names (HTML)
<code>@RestController</code>	REST API	<code>@Controller</code> + <code>@ResponseBody</code> — returns JSON automatically
<code>@Configuration</code>	Config	Declares <code>@Bean</code> factory methods
<code>@Bean</code>	Config Method	Produces a bean inside a <code>@Configuration</code> class

@SpringBootApplication

The main entry-point annotation — a shortcut that combines three annotations:

- `@Configuration` — this class is a configuration source
- `@EnableAutoConfiguration` — enables Spring Boot auto-configuration
- `@ComponentScan` — scans the package for annotated components

```
@SpringBootApplication
public class MyApp {
    public static void main(String[] args) {
        SpringApplication.run(MyApp.class, args);
    }
}
```

4. Spring MVC & REST API

Servlet Container & DispatcherServlet

Spring Boot ships with an **embedded Tomcat** server. All HTTP requests flow through a single **DispatcherServlet** (Front Controller), which routes requests to the right `@RestController` method.

Core REST Annotations

Annotation	HTTP Method	Purpose
<code>@GetMapping</code>	GET	Retrieve / read data — no side effects
<code>@PostMapping</code>	POST	Create a new resource

Annotation	HTTP Method	Purpose
@PutMapping	PUT	Fully replace an existing resource
@PatchMapping	PATCH	Partially update an existing resource
@DeleteMapping	DELETE	Delete a resource
@RequestMapping	ANY	Base path mapping on class level

Request Handling Annotations

Annotation	Source	Example URL
@PathVariable	URL path segment	/students/{id} → /students/42
@RequestParam	Query string	/students?name=Alice&page=1
@RequestBody	Request body (JSON)	POST body: { "name": "Alice" }
@RequestHeader	HTTP header	Authorization: Bearer <token>

ResponseEntity — Full Control

Gives you control over status code, headers, and body:

```
return ResponseEntity.status(HttpStatus.CREATED).body(saved);    // 201
return ResponseEntity.ok(student);                               // 200
return ResponseEntity.noContent().build();                       // 204
return ResponseEntity.notFound().build();                        // 404
```

HTTP Status Codes

Code	Constant	Meaning
200	HttpStatus.OK	Request succeeded
201	HttpStatus.CREATED	Resource created successfully
204	HttpStatus.NO_CONTENT	Success, no response body
400	HttpStatus.BAD_REQUEST	Invalid client request
401	HttpStatus.UNAUTHORIZED	Authentication required
403	HttpStatus.FORBIDDEN	Authenticated but not authorized
404	HttpStatus.NOT_FOUND	Resource not found
500	HttpStatus.INTERNAL_SERVER_ERROR	Server-side error

Global Exception Handling

```
@RestControllerAdvice
public class GlobalExceptionHandler {
```

```
@ExceptionHandler(ResourceNotFoundException.class)
public ResponseEntity<String> handleNotFound(ResourceNotFoundException ex) {
    return ResponseEntity.status(HttpStatus.NOT_FOUND).body(ex.getMessage());
}

@ExceptionHandler(Exception.class)
public ResponseEntity<String> handleGeneral(Exception ex) {
    return ResponseEntity.internalServerError().body("Something went wrong");
}
}
```

5. JPA, Hibernate & Spring Data JPA

The Problem with Raw JDBC

With JDBC you must manually write SQL for every query, map each `ResultSet` column to a Java field, and handle all CRUD boilerplate — very repetitive.

What is ORM?

ORM (Object-Relational Mapping) bridges the gap between **Java Objects** and **Database Tables** automatically. You think in Java; Hibernate handles the SQL.

The Abstraction Stack

Layer	What It Is
Spring Data JPA	Highest abstraction — magic repositories with zero SQL
JPA API	Standard specification / set of interfaces (not runnable code)
Hibernate	The actual JPA implementation — generates and runs SQL
JDBC	Talks directly to the database

JPA Entity

```
@Entity
@Table(name = "products")
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private double price;
    // getters and setters
}
```

JpaRepository — Zero SQL CRUD

```
public interface ProductRepository extends JpaRepository<Product, Long> {
    // findById, save, findAll, deleteById — all built-in!
    List<Product> findByName(String name); // derived query — no SQL needed
}
```

Common JPA Annotations

Annotation	Purpose
@Entity	Marks class as a JPA entity (maps to a DB table)
@Table(name=...)	Specifies the exact table name
@Id	Marks the primary key field
@GeneratedValue	Auto-generates the ID (IDENTITY, SEQUENCE, AUTO)
@Column(name=...)	Maps field to a specific column name
@OneToMany / @ManyToOne	Defines relationships between entities
@Transactional	Wraps a method in a DB transaction
@Lob	Maps large objects (CLOB/BLOB) — use with caution in H2

Connection Pooling (HikariCP)

Opening a new DB connection for every request is slow and resource-heavy. Spring Boot auto-configures HikariCP — a pool of pre-opened, reusable connections.

```
# application.properties
spring.datasource.hikari.maximum-pool-size=10
spring.datasource.hikari.minimum-idle=2
spring.datasource.hikari.connection-timeout=30000
```

6. application.properties / application.yml

The central config file for a Spring Boot app — lives in src/main/resources/.

Common Properties

```
# Server
server.port=8080

# H2 In-Memory Database
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driver-class-name=org.h2.Driver
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

# MySQL Database
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root
spring.datasource.password=secret
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# JPA / Hibernate
spring.jpa.hibernate.ddl-auto=update # create | create-drop | validate | update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# Logging
logging.level.org.springframework=INFO
logging.level.com.yourpackage=DEBUG
```

ddl-auto Value	Behaviour
create	Drops and recreates schema on every startup — wipes data!
create-drop	Creates on start, drops on shutdown — for testing
update	Applies schema changes without dropping existing data
validate	Validates schema but makes no changes — for production
none	No schema management at all

7. Spring Security

The Security Filter Chain

Every HTTP request passes through a chain of security filters before reaching your controller:

```
HTTP Request
→ UsernamePasswordAuthenticationFilter (extracts credentials)
→ AuthenticationManager
→ AuthenticationProvider ← calls UserDetailsService + BCryptPasswordEncoder
→ SecurityContext (stores authenticated user for this request)
→ Your @RestController
```

SecurityFilterChain Configuration

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .csrf(csrf -> csrf.disable()) // disable for JWT APIs
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/auth/**").permitAll()
            .requestMatchers("/api/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated()
        )
        .sessionManagement(s ->
            s.sessionCreationPolicy(SessionCreationPolicy.STATELESS) // no sessions
        )
        .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
    return http.build();
}
```

UserDetailsService

```
@Service
public class MyUserDetailsService implements UserDetailsService {
    @Override
    public UserDetails loadUserByUsername(String username) {
        User user = userRepo.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException("Not found"));
        return new org.springframework.security.core.userdetails.User(
            user.getUsername(), user.getPassword(), getAuthorities(user));
    }
}
```

Password Encoding — BCrypt

Passwords must **NEVER** be stored as plain text. Always hash with BCrypt:

```
BCryptPasswordEncoder encoder = new BCryptPasswordEncoder(12);
String hashed = encoder.encode("myPassword123");
boolean match = encoder.matches("myPassword123", hashed); // true
```

	Encryption	Hashing
Reversible?	Yes (with key)	No — one-way only
Use case	Sensitive data in transit/storage	Passwords
Example	AES, RSA, HTTPS	BCrypt, SHA-256

❑ *Passwords should NEVER be encrypted — they should be HASHED. Encryption can be reversed; hashing cannot.*

Role-Based Access Control

```
@PreAuthorize("hasRole('ADMIN')")
@GetMapping("/admin/users")
public List<User> getAllUsers() { ... }
```

```
@PreAuthorize("hasAnyRole('ADMIN', 'MANAGER')")
@GetMapping("/reports")
public Report getReport() { ... }
```

CSRF — Cross-Site Request Forgery

CSRF is an attack where a malicious site tricks your browser into making unwanted requests using your stored session cookie. **Spring Security enables CSRF protection by default** for state-changing operations.

❑ *For REST APIs using JWT (stateless), CSRF is not a concern — disable it with `csrf.disable()`. CSRF only matters for session/cookie-based auth.*

8. JWT Authentication

What is JWT?

JWT (JSON Web Token) is a compact, self-contained token for stateless authentication. A JWT has 3 parts separated by dots: **Header.Payload.Signature**

```
eyJhbGciOiJIUzI1NiJ9 . eyJzdWIiOiJhbGljZSJ9 . SflKxwRJSMeKKF2QT4fwpMeJf36
Header (alg) Payload Signature
```

JWT Flow

- Client sends username + password to POST /api/auth/login
- Server validates credentials, generates a signed JWT, returns it
- Client stores token (localStorage / memory)
- Every subsequent request includes: Authorization: Bearer <token>
- JwtFilter intercepts the request, validates the token, sets SecurityContext
- Controller handles the request

JWT Components

Class	Responsibility
JwtService	Generate token, extract claims, validate expiry and signature
JwtFilter	Intercepts every request, extracts token from header, authenticates
AuthController	POST /login endpoint — validates user, calls JwtService, returns token
SecurityFilterChain	Registers JwtFilter before UsernamePasswordAuthenticationFilter

JwtService Example

```
public String generateToken(String username) {
    return Jwts.builder()
        .setSubject(username)
        .setIssuedAt(new Date())
        .setExpiration(new Date(System.currentTimeMillis() + 1000 * 60 * 60 * 10))
        .signWith(getSignKey(), SignatureAlgorithm.HS256)
        .compact();
}
```

Signing Algorithm	Type	Use Case
HS256	Symmetric (same secret for sign + verify)	Single-service apps
RS256	Asymmetric (private key signs, public key verifies)	Microservices, distributed

9. JDBC — Raw Database Access

The 7 Steps of JDBC

Step	What Happens
1. Load Driver	Registers the DB-specific JDBC driver with the JVM
2. Define Connection URL	<code>jdbc:mysql://localhost:3306/mydb</code> — location of the database
3. Establish Connection	<code>DriverManager.getConnection(url, user, password)</code>
4. Create Statement	<code>conn.prepareStatement(sql)</code> — creates the query executor
5. Execute Query	<code>executeQuery()</code> for SELECT; <code>executeUpdate()</code> for INSERT/UPDATE/DELETE
6. Process Result	Iterate <code>ResultSet</code> row by row to read returned data
7. Close Connection	Always release the connection — use try-with-resources

PreparedStatement Example

```
Connection conn = DriverManager.getConnection(url, user, password);
PreparedStatement ps = conn.prepareStatement("SELECT * FROM products WHERE id = ?");
ps.setInt(1, 101);
ResultSet rs = ps.executeQuery();
while (rs.next()) {
    System.out.println(rs.getString("productName"));
}
conn.close();
```

Statement Type	Use When	SQL Injection Safe?
Statement	SQL has no user input (static queries)	No
PreparedStatement	SQL contains user-provided values	Yes — use always

❑ *Raw JDBC is verbose and error-prone. In Spring Boot, use Spring Data JPA — it wraps JDBC with Hibernate and gives you repositories for free.*

10. Plain Spring — XML Configuration

Before Spring Boot, all bean configuration was done manually via XML files.

Maven Dependency (plain Spring)

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-context</artifactId>
  <version>6.1.6</version>
</dependency>
```

spring.xml — Bean Definitions

```
<beans xmlns="http://www.springframework.org/schema/beans" ...>

    <bean id="dev" class="com.example.Dev">
        <property name="age" value="12" />           <!-- setter injection -->
        <property name="laptop" ref="lap1" />         <!-- object injection -->
    </bean>

    <bean id="lap1" class="com.example.Laptop" />
</beans>
```

Loading the Container

```
ApplicationContext context = new ClassPathXmlApplicationContext("spring.xml");
Dev obj = context.getBean(Dev.class);    // by type
Dev obj = (Dev) context.getBean("dev");  // by id
```

Injection Type	XML Tag	Java Requirement
Setter Injection	<property name="x" value=".." />	setX() method required
Constructor Injection	<constructor-arg value=".." />	Matching constructor required
Object Injection (setter)	<property name="x" ref="beanId" />	setX() method required
Object Injection (constructor)	<constructor-arg ref="beanId" />	Matching constructor required

□ *In Spring Boot, ALL of this is replaced by @Component, @Autowired, and auto-scanning. XML config is rarely used in modern projects.*

11. Spring Profiles & Actuator

Spring Profiles

Profiles let you have different configurations for different environments (dev, test, prod).

```
# application-dev.properties
spring.datasource.url=jdbc:h2:mem:devdb

# application-prod.properties
spring.datasource.url=jdbc:mysql://prod-server:3306/mydb
```

```
# Activate a profile in application.properties
spring.profiles.active=dev
```

```
@Profile("dev")
@Component
public class DevDataLoader { ... } // Only created in dev profile
```

Spring Boot Actuator

Actuator exposes production-ready monitoring endpoints. Add to pom.xml:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Endpoint	URL	What It Shows
/health	/actuator/health	App health status (UP/DOWN)
/info	/actuator/info	App info (version, description)
/metrics	/actuator/metrics	JVM, memory, request metrics
/env	/actuator/env	All environment properties
/beans	/actuator/beans	All Spring beans in context
/mappings	/actuator/mappings	All @RequestMapping routes

12. Encryption, Hashing & Security Concepts

Encryption

Encryption converts plain text → cipher text using a **key**. Only someone with the key can reverse it.

Type	How It Works	Example
Symmetric	Same key for encrypt & decrypt	AES — fast, for data at rest
Asymmetric	Public key encrypts, private key decrypts	RSA, HTTPS/TLS
Hashing	One-way — CANNOT be reversed	BCrypt — for passwords

HTTPS & JWT Signing

JWT tokens use signing (not encryption) to ensure the token hasn't been tampered with:

- HS256 — Symmetric: same secret key to sign and verify
- RS256 — Asymmetric: private key signs, public key verifies (safer for microservices)

Authentication Provider Flow

```
Unauthenticated Object (username + password)
  → AuthenticationProvider
    → UserDetailsService (loads user from DB)
    → BCryptPasswordEncoder (checks password hash)
  → Authenticated Object (user + roles/authorities)
  → Stored in SecurityContext for this request
```

Quick Reference Cheat Sheet

Common Maven Dependencies

```
<!-- Web + REST API -->
<dependency>spring-boot-starter-web</dependency>

<!-- JPA + Hibernate -->
<dependency>spring-boot-starter-data-jpa</dependency>

<!-- Security -->
<dependency>spring-boot-starter-security</dependency>

<!-- H2 In-Memory DB -->
<dependency>h2 (runtime scope)</dependency>

<!-- JWT -->
<dependency>io.jsonwebtoken:jjwt-api</dependency>
<dependency>io.jsonwebtoken:jjwt-impl (runtime)</dependency>
<dependency>io.jsonwebtoken:jjwt-jackson (runtime)</dependency>
```

Architecture Layers

Layer	Class Type	Annotation	Responsibility
Web	Controller	@RestController	Handle HTTP, validate input, return JSON
Business	Service	@Service	Business logic, orchestration
Data	Repository	@Repository / JpaRepository	DB queries, data access
Model	Entity/DTO	@Entity	Data structure (table mapping or transfer)

Annotation Cheat Sheet

Annotation	Where	What It Does
@SpringBootApplication	Main class	Combines @Configuration + @EnableAutoConfiguration + @ComponentScan
@RestController	Controller class	Returns JSON from all methods
@RequestMapping("/api")	Class level	Base path for all routes in class
@GetMapping / @PostMapping...	Method level	Maps HTTP method to handler
@PathVariable	Method param	Extracts /users/{id} → Long id
@RequestParam	Method param	Extracts ?page=1 → int page
@RequestBody	Method param	Deserializes JSON body to Java object

Annotation	Where	What It Does
@Autowired	Field/Constructor	Injects bean from Spring container
@Qualifier("name")	Alongside @Autowired	Specifies which bean to inject
@Entity	Model class	Maps class to DB table
@Id + @GeneratedValue	Entity field	Primary key with auto-increment
@Transactional	Service method	Wraps method in DB transaction
@PreAuthorize("hasRole")	Controller method	Method-level security check
@ExceptionHandler	Method in advice	Handles a specific exception type
@RestControllerAdvice	Class	Global exception handler for all controllers
@Profile("dev")	Class/Bean	Only activate in specified profile
@Value("\${prop.key}")	Field	Injects value from application.properties